We talk about fast protocols, but the client has to be semi-honest: two Attacks and a Mitigation for Leap

Anonymous authors

No Institute Given

Abstract. The Leap OPRF was introduced with a security proof in the semi-honest model, leaving its security against malicious adversaries as an open question. In this work, we provide the first security analysis of Leap in a malicious-client setting and present a full key recovery attack against both modes. The polynomial-complexity atttack recovers the complete server key in fewer than 150 000 OPRF evaluations, demonstrating a critical vulnerability for practical deployment. Building on this analysis, we propose a new combined mode that combines both modes that is proven in the semi-honest model and resists our attacks. Although we do not provide a proof in the malicious model, we identify and analyze the key obstacles to achieving malicious security for our new construction. In addition, the new construction seems to indicate that it is possible to reduce modulus in Leap to as few as four bits, which halves the communication complexity in the online phase.

Keywords: Leap· Naor-Reingold OPRF · lattice-based cryptography · cryptanalysis

1 Introduction

Several presumably post-quantum secure Oblivious Pseudorandom Functions (OPRFs) have been proposed in the recent years as secure alternatives to elliptic-curve based constructions in the presence of quantum adversaries. Among these, the Leap OPRF stands out by obliviously evaluating the lattice-based Naor-Reingold PRF Spring, diverging from the more commonly used 2HashDH paradigm. While Leap is very fast, its security is not as well analyzed as other constructions: The underlying PRF is not provably secure, and the OPRF is only proven secure in the semi-honest model.

In this work, we study Leap in the malicious setting. We identify two distinct attacks on the OPRF in the malicious model, one for the well-described BCH mode and one for the CRT mode, which until now has only been sketched. We show that the final part of the protocol, the oblivious rounding step, does not sufficiently hide the server private key, and recover the full key in $(\kappa + 1)n \log q$ steps for the BCH mode and in $(\kappa + 1) \log q$ steps for the CRT mode. As a part of this analysis, we also provide the first explicit description of the CRT mode for Leap.

While the attacks do not contradict the semi-honest security proof of Leap, they highlight a critical weakness that makes practical deployment impossible. To address the attacks, we introduce a new mode of operation: Leap-CB combines the CRT mode and the BCH mode. The new mode effectively doubles the modulus, but is resistant against the attacks with very little additional overhead. We adapt the original proof of Leap to establish security of the new mode and provide a detailed analysis. However, the protocol still leaks partial information about the server private key, allowing an adversarial client to distinguish the output from a random function. This prevents a fully malicious secure construction.

Interestingly, the leakage also suggests a new direction. Since all polynomial coefficient bits but the most significant bit are revealed, we show that the modulus can be drastically reduced- to as little as four bits- without compromising security in the semi-honest setting. We conclude by outlining concrete parameterizations for such lightweight variants of Leap, leaving their cryptanalysis as future work.

2 Preliminiaries

2.1 SPRING

The lattice PRF Spring [BBL+15] is an efficient instantiation of the BPR12 Naor-Reingold PRF [BPR12] over the polynomial ring $R := \mathbb{Z}[X]/(X^n + 1)$. The PRF key $\mathbf{K}$ consists of κ polynomials $\mathbf{K} = (\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_\kappa)$ that represent ring elements. To compute the PRF for κ input bits $\mathbf{c} = (c_1, \ldots, c_\kappa) \in \mathbb{Z}_2^\kappa$, each of the ring elements $\mathbf{k}_i$ of the key is multiplied with $\mathbf{k}_0$ if the ith bit c_i is set. The multiplication output is then rounded with the rounding function S.

$$\textsc{Spring}(\mathbf{c}, \mathbf{K}) = S\left(\mathbf{k}_0 \cdot \prod_{i=1}^{\kappa} \mathbf{k}_i^{c_i}\right)$$

Spring is significantly faster than the [BPR12] PRF due to two modifications of the parameters: First, the BPR12 PRF $\lfloor \mathbf{a} \prod \mathbf{s}_i \rceil$ takes a secret, uniformly random binary input $\mathbf{a} \in \mathbb{R}_q$ and samples $s \in \mathbb{R}_q$ from independent discrete Gaussians. The ring-LWR distribution is over $\mathbb{R}_q \times \mathbb{R}_p$, where q has to be exponentially large in the input length. Spring relaxes the requirement to a polynomially large modulus q. Second, the keys are drawn from an uniform distribution in Spring, while BPR12 is that the keys $\mathbf{k}_i$ are small ring elements in $\mathbb{R}$ drawn from the error distribution of the underlying Ring-LWE assumption. These changes imply that the reduction from Spring to Ring-LWR is not tight, so breaking Spring does not necessarily mean that Ring-LWR is broken.

The key polynomials $\mathbf{k}_i$ *must be units.* The polynomials used in the lattice PRF have to be invertible to avoid an attack that significantly reduces the entropy

of the product when $\mathbf{K}$ is large relative to q: If the coefficients are drawn uniformly at random, a coefficient is zero with a probability of $\frac{1}{q}$. Multiplying many polynomials together may result in the multiplications eventually cancelling out. This can lead to multiple inputs evaluating to the same output, and void the pseudorandomness guarantees of the PRF. To avoid this attack, q must either be large enough so that this is unlikely, or the polynomials have to be units. A polynomial is a unit if the GCD with the modulus polynomial is 1. A unit is invertible, which is very useful for provable security, since it helps with the extractability of the original values in a simulation. Finally, invertible polynomials are important for oblivious evaluation, as they enable unblinding previously blinded values.

For $q = 257$, a polynomial is a unit if its NTT has no zero coefficients. [BPR12] mention that restricting $\mathbf{K}$ to only contain units seems to be the most efficient way to mitigate the cancelling out. Restricting the key polynomials to units has not resulted in any attacks on the scheme at the time of writing. The Spring paper claims attacking Spring is harder than attacking LWE, because ring-LWE releases the multiplicand a without any error, while all terms of Spring have an error. Concretely, spring rounds each polynomial coefficient z_i of the multiplication result from $\mathbb{Z}_q$ to $\mathbb{Z}_2$. The most common rounding function used for the PRF modes of Spring is $\begin{cases} 1 & \frac{q}{4} \leq z_i < \frac{3q}{4} \\ 0 & otherwise \end{cases}$. For the odd modulus $q = 257$, we need to apply the $[128, 64, 22]$ BCH extension code to mitigate the rounding bias.

Finally, restricting the key to invertible polynomials enables a neat implementation trick: instead of a subset product $\mod q$, we compute a subset-sum $\mod q - 1$. This requires a generator g, which is 3 for Spring. Each coefficient is now expressed as a power of g. For example, 9 becomes 2 since $3^2 \equiv 9$. Since multiplication turns into exponent addition, we effectively reduced multiplication to addition: For example, instead of computing the product $9 * 27 = 243$, we can rewrite the terms as their logarithm and add the exponents $(3^2) + (3^3) = 3^5$. The exponent representation needs to be converted back to get the final result, either by computing the exponentiation or performing a table lookup. Since $a^{q-1} \equiv a^0 \equiv 1$, a direct mapping is ambiguous. However, since the polynomials are restricted to nonzero coefficients, we consistently map to $q - 1$. This allows uniform sampling in logarithmic form, to which we refer as subset-sum form throughout the paper. As an added benefit, the coefficients fit neatly into one byte for the parameters of Spring-BCH.

So far, three modes have been introduced for Spring. The original paper introduces Spring-BCH, with an uneven modulus q, and Spring-CRT, with an even modulus and a better rate. Spring-CRT can optionally also be used with a counter mode (Spring-CTR). Spring-RS [BDFK17] extends Spring to a PRNG. We now give a brief introduction to each mode and its applicability to OPRFs.

Spring-BCH The uneven modulus $q = 257$ has a few advantages in the implementation, but biases the output coefficients with a rounding bias of $\frac{1}{257}$. To mitigate the bias, Spring calls the $[128, 64, 22]$ extended BCH code, which reduces the bias to $\frac{1}{q^d} = 2^{-177}$. As a result, the output is halved. For 128 bits of input, Spring-BCH returns 64 bits of output.

Spring-CRT In the Chinese Remainder Theorem (CRT) mode, the modulus $\bar{q} = 2q = 2 * 257 = 514$ is used to decompose the subset-product computation into its Chinese remainder. For R^*_{257}, the same evaluation strategy as in Spring-BCH is used, but the decomposition of R_2 is different, as it decomposes into n/2 small cyclic groups with power-of-two orders of at most n to transform from a subset-product into a subset-sum once again. The R_2 polynomials are a unit when the sum of their coefficients is odd. To reliably get a polynomial that is a unit, the first $n-1$ coefficients are sampled at random, and the last coefficient is 1 if the sum of the random coefficients is even and 0 otherwise. This adds a bit more structure to the polynomial again.

Spring-CRT also supports arbitrarily long inputs by using a *counter-like* mode. The *CTR mode* aims to amortize the computational work of the subset-product by sorting the input blocks in a Gray code. This avoids recomputing the full product from scratch for every input block and makes the computation more efficient.

Sublinear combinatorial attack Spring-CRT is susceptible to a sublinear combinatorial attack. The original Spring paper presents an effective distinguisher that aims to eliminate the R_2 component of Spring-CRT. Once the R_2 component is cancelled out, the adversary can use the rounding bias of the remaining R_{257} component as an effective distinguisher.

The rounding result $\lfloor b \rceil_2$ of a polynomial coefficient b is the sum of the rounding results for each component, such that $b \equiv q\bar{b_2} + q\bar{b_q} + \bar{b_q} \equiv q\bar{b_2} + (q+1)\bar{b_q} \mod 2q \equiv \bar{b_2} + \bar{b_q} + \frac{\bar{b_q}}{q} \mod 2$. Since the coefficients of $\bar{b_q}$ are chosen from $[\frac{-q}{2}, \frac{q}{2}) \cap \mathbb{Z}$, $\frac{\bar{b_q}}{q}$ cancels out as it is in the interval $[\frac{-1}{2}, \frac{1}{2})$ [BBL+15, Theorem 3]. As a result, $\lfloor b \rceil_2 \equiv \bar{b_q} + \bar{b_2} \mod 2$.

Cancelling out the R_2 component $\bar{b_2}$ of the rounding result results in a biased output. The bias can then be used to recover the secret key. To remove the R_2 component, the input $\mathbf{x}$ is split into two parts $\mathbf{y}, \mathbf{z}$ such that $\mathbf{x} = \mathbf{y}||\mathbf{z}$. Then, Spring-CRT can be written as $F(\mathbf{y}||\mathbf{z}) = \lfloor \mathbf{k}_0 \cdot \mathbf{s_y} \cdot \mathbf{s_z} \rceil_2$, where $\mathbf{s_y}$ and $\mathbf{s_z}$ represent the subset product for the keys $\mathbf{k}_i$ indicated by the bits of $\mathbf{y}$ and $\mathbf{z}$. The adversary first draws samples until it finds the values $\mathbf{y}$ and $\mathbf{y}' \neq \mathbf{y}$ such that $\mathbf{s_y} = \mathbf{s_{y'}}$. With high probability, this takes at most $2 * 2^{n/2}$ samples. Then, the bias can be detected using $\frac{q^4}{4n}$ output bits with varying values of z. The original paper estimates the query and space complexity to be $2^{\frac{n}{2}} q^8/(4n)$. For the parameters $n = 128, q = 514$, the attack takes 2^{126} operations and 2^{119} space.

The follow-up paper that introduces Spring-RS [BDFK17] proposes a constant-time approach to find collisions. Instead of comparing all values with each other

to find a statistical bias, the correlations are computed with matrix multiplications. This significantly speeds up the bias detection step. Computationally, the attack requires 2^{130} operations and is a bit more computationally expensive, Strassen's algorithm achieves a constant distinguishing advantage.

Note that we can mitigate a concrete attack by increasing the polynomial degree or the modulus.

Spring-RS Follow-up work on Spring introduces *Spring with Rejection Sampling* (Spring-RS) [BDFK17], a mode of Spring with similar parameters $q = 257$ to the Spring-BCH mode, with the additional $p \in \{2, 4, 8, 16\}$. The rounding function S_{coef} rounds each coefficient b_i to its $\log_2(p)$ high-order bits. The case where $b_i = 256$ is *rejected*, and no output is returned. The rejection mitigates the rounding bias, but now means that Spring-RS has a variable output length.

Extracting more than one bit is possible due to the high attack complexity for the Spring parameters that have also been discussed in [HKL+25], where the best attack for the parameters of Spring-BCH had a complexity of $2^{503.3}$ for a Gröbner basis attack [AG11]. The authors of Spring-RS computed 900 bits of security with binary rounding. As a result, it is acceptable to extract more than one bit per coefficient by rounding to a larger set. The main use case for Spring-RS is a PRG, where slightly irregular output lengths are acceptable. Building a PRF from rejection sampling is more difficult, as it requires a fixed output length. The output for the Spring-RS PRF is repeatedly computed until 96 coefficients have not been rejected. [1] Since the previously mentioned CTR mode is used, this is possible for all inputs.

Leap does not discuss Spring-RS as a mode for oblivious evaluation. It turns out not returning any rounding result, or $\perp$, is impossible to capture in the OT without leaking that $b_i = 256$, which occurs in 0.4% of all coefficients. However, Spring-RS motivates our construction to *lower the modulus*, described in Section 6.

2.2 Leap

Leap is a lattice-based Oblivious Pseudorandom Function (OPRF) that obliviously evaluates the Spring PRF between a client that holds the input $\mathbf{x}$ and a server that holds the input $\mathbf{K}$. The OPRF is secure if the server learns nothing, especially nothing about the client's input, and the client only learns the output, but strictly nothing about the server key. Leap is proven secure against semi-honest adversaries in the Universal Composability (UC) model. For the oblivious evaluation, Leap relies heavily on one-out-of-two Oblivious Transfer ($\binom{2}{1}$-OT) as a core building block. To get a more efficient online phase, the OT computations are preprocessed using base OT [Bea96], enabling an efficient online phase

[1] The erasure process may yield fewer than 96 coefficients. This happens with a probability of 2^{-156}. In such cases, the PRF output is padded with zeros. This padding is effectively undetectable to a computationally bounded adversary due to the extremely low likelihood of the event.

which primarily consists of lightweight OT corrections and simple arithmetic operations. The preprocessing phase is made more efficient using OT extensions [BCG+19] that generate polynomially many more OT pairs with sublinear communication from a small number of OTs. Since the preprocessing phase scales well with the number of inputs, Leap is a good choice for protocols that perform many OPRFs, such as private set intersection protocols, as an evaluation of Leap takes less than a millisecond.

Leap-BCH The original Leap paper only describes the BCH mode. To hide the rounding bias, each rounding output bit is masked with a random bit d_i. To remove the blinding elements $\mathbf{d}$ from the client output, the server reveals $\mathsf{BCH}(\mathbf{d})$. Since $\mathsf{BCH}(\mathbf{e}) \oplus \mathsf{BCH}(\mathbf{d}) \equiv \mathsf{BCH}(\mathbf{e} \oplus \mathbf{d})$, the client can remove the blinding by computing a BCH code over the rounding results and xoring them with $\mathsf{BCH}(d)$.

Leap-CRT The CRT mode is not explicitly described in the original paper [HKL+25], since the main aim was constructing an efficient OPRF for private set intersection. However, the authors claim in the presentation of the paper at Eurocrypt 2025 that the evaluating Leap with Spring-CRT should work analog to the BCH mode sketched in Figure 1, without sampling the blinding elements $\mathbf{d}$ and sending the BCH code. We explicitly discuss the CRT mode, especially the polynomial decomposition, in Section 3.

2.3 Leap is susceptible to bit-by-bit leakage through a malicious server.

Naor-Reingold OPRFs may leak the client input to a malicious server for repeated executions when the output of the Naor-Reingold OPRF is revealed to the server and the server is not forced to use the same key in each OPRF execution [JKK14]. To leak the client input, the server can fault the blinded key input to represent another element $r_i' \neq r_i \neq r_i k_i$. If the output is revealed later and the server can check the validity of the input, for example observing in an OPRF-based password authentication protocol like OPAQUE, whether a login attempt is successful or not, they know the bit is set if the authentication fails, and is not set when the authentication is successful.

Leapis only proven secure in the semi-honest model, so the attack does not violate the security claims of the paper. For elliptic curves, the attack is mitigated by adding *verifiability* to the OPRF. The server has to commit to its keys $\mathbf{K}$ before the OPRF execution and prove they were used to compute the OPRF output. This mitigation does not translate to a post-quantum setting so far. Both Leap and the CSIDH-based Naor-Reingold OPRFs OPUS and NR-OT [HHM+24] do not offer verifiability and are susceptible to a malicious server attack.

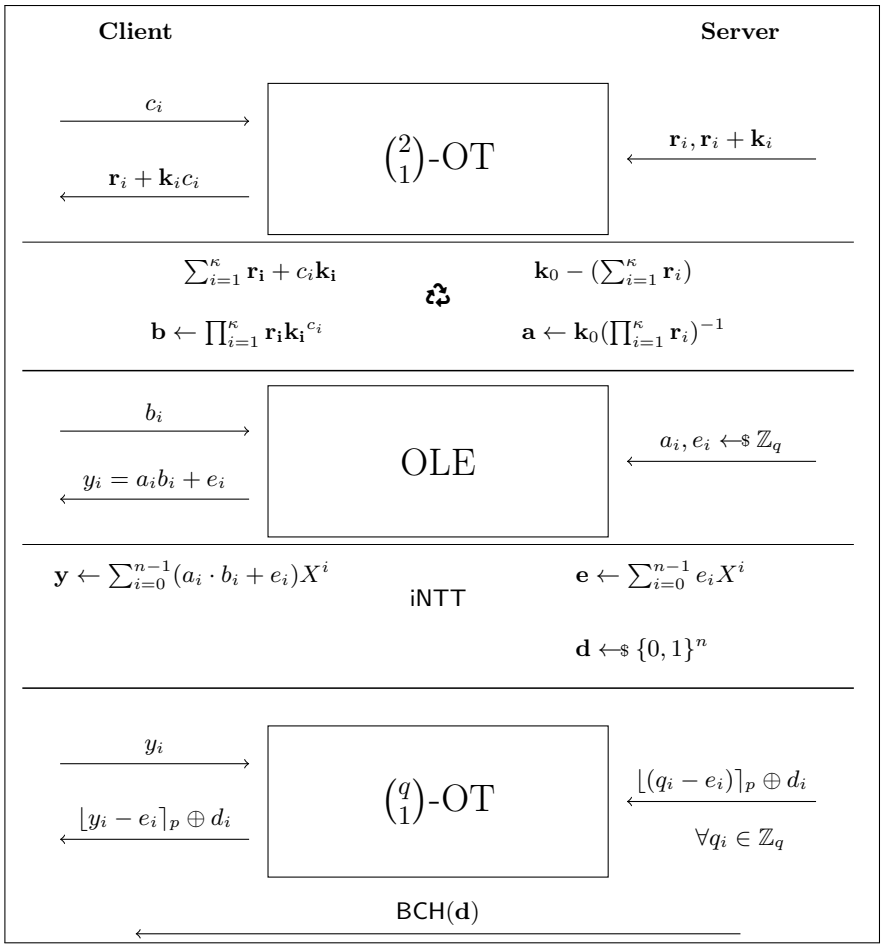

Fig. 1: Full protocol of Leap. The BCH function call and sampling of $\mathbf{d}$ are only required in Leap-BCH.

Relevance on protocols [CHL22] divide the role of OPRFs in a protocol into two categories: (1) retrieval of cryptographic material, and (2) enforcing interaction when computing hash values, e.g. for rate limiting. LEAP is optimized for private set intersection, where the output is not necessarily revealed to the server and repeated OPRF executions with the same input are uncommon. We refine this categorization by highlighting two scenarios under which a NR-OPRF can still be used without compromising the security of the protocol.

1. The OPRF output is never revealed to the server (PSI).
2. The OPRF input is randomized (PrivacyPass).

The first condition can sometimes be relaxed. If the same input is queried only a limited number of times, only a few bits of information may be leaked. Moreover, a practical attack may be difficult to carry out. For instance, in password-based protocols a failure caused by fault injection is indistinguishable from a simple typo, making such attacks harder to detect on the first attempt. Nevertheless, the overall impact is application-dependent, and we recommend avoiding Naor-Reingold OPRFs when the threat model is unclear.

Protocol	output revealed	input randomized	repeated execution	can use NR-OPRF
OPAQUE [JKX18]	✓	✗	✓	✗
Obliv. Keyword Search [FIPR05, KKRT16]	✗	✗	✗	✓
Private Set Intersection [HL08, KRS+19]	✗	✗	✗	✓
Password-Protected Secret Sharing [JKK14]	✓	✗	✓	✗
PAKE [JKK14]	✓	✗	✓	✗
private SSO [BFH+20]	✓	✗	✓	✗
cloud key management [JKR19]	✓	✗	✓	✗
secure pattern matching [HL08]	✓	✗	✗	
CAPTCHA [CDFH+22]	✓	✓	✗	✓
Private SSO [KLÖ25]	✓	✗	✓	✗

Fig. 2: Protocols built on OPRFs and whether they can employ non-verified Naor-Reingold OPRFs.

One possible mitigation is to use a *verifiable* Naor-Reingold OPRF, though this may be prohibitive due to the large number of keys required. Alternatively, server-side safeguards or policy constraints may be sufficient to deter attacks, particularily when the adversaries face the risk of reputational damage or regulatory fines.

2.4 Notation

Our notation follows [HKL+25] closely. We write integers and generic variables like g in math mode, vectors like $\mathbf{v}$ in lowercase bold, the ith vector index as v_i, and matrices like $\mathbf{M}$ in uppercase bold. The matrix of key polynomials is $\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots \mathbf{k}_\kappa] \in \mathbb{Z}_q^{n \times (\kappa+1)}$. The user input is always $\mathbf{c} \in \{0,1\}^\kappa$. The modulus is $q \in \{514, 257\}$, and rounds to $p = 2$. The security parameter is κ. The polynomial dimension is $n = 128$. We will denote the basis vectors $\mathbf{e}_i \in \{0,1\}^\kappa$, where the ith index is one and all other vector indices are 0.

3 LEAP-CRT

When studying LEAP, we observe that the LEAP-CRT construction cannot follow exactly the same design principles as LEAP-BCH. This is due to the CRT decomposition of the polynomial ring $R_{2q} \to R_2 \times R_q$. The R_q component can be computed exactly as in LEAP-BCH, and the subset-sum of the R_2 component can be computed similarly to the R_q component. However, a straightforward evaluation requires a later *recombination* of the decomposition, which can be performed in the rounding step. To clarify these changes, Figure 3 provides a complete overview of the adapted LEAP-CRT protocol that processes the R_q and the R_2 components in parallel.

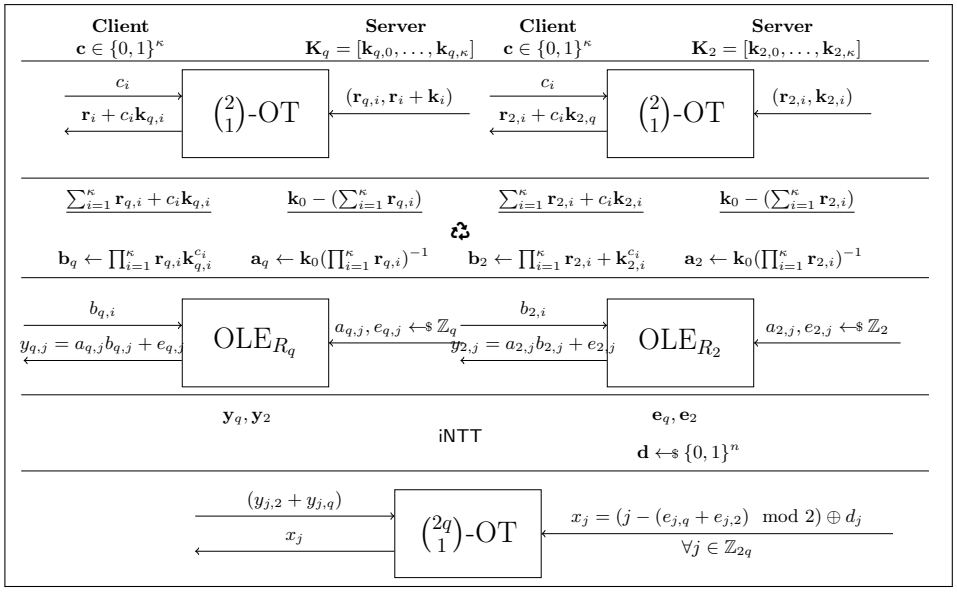

Fig. 3: Naïve depiction of LEAP-CRT. We relabel $\mathbf{k}_0$ from the original LEAP protocol to $\mathbf{k}_\kappa$, which simplifies notation and offsets without impacting correctness, as the order is irrelevant for the protocol logic. The client input $\mathbf{c}$ is the same for both client sides, but was repeated to show the potentially parallel computation across both R_q and R_2.

When recombining the R_q and R_2 component, we have to ensure the R_q component is first reduced to the range $[\frac{-q}{2}, \frac{q}{2})$ before adding the R_2 component. This ordering is necessary to avoid incorrect results due to overflows in the modular computation. Consider the following example with $y_q = 5, e_q = 255, e_2 = 1$ and $y_2 = 0$. Summing up all integers, we get $y_q + e_2 + e_q + y_2 = 301$. However, if we compute first in $\mathbb{Z}_q$, $y_q + e_q = 300 \equiv 3 \mod 257$ so the actual correct

result when adding the R_2 component is $3 + 1 = 4 \equiv 0 \mod 2$, and not $301 \equiv 1 \mod 2$.

3.1 Combining the OT steps

The straightforward oblivious computation processes the R_2 and the R_q component separately. However, since the clients input bits are identical across both computations and the decomposition is a linear mapping, it should be possible to combine the subset-sum $\binom{2}{1}$-OT computations in a single operation. Nevertheless, in the online phase the R_2 component is only adding $n\kappa$ bits to the computation, rendering the potential performance gain marginal. For this reason, we leave this optimization for future work.

4 Full key recovery attack against Leap

The OPRF Leap [HKL+25] obliviously evaluates the PRF Spring. More concretely, it obliviously evaluates the Naor-Reingold PRF $S\left(\mathbf{k}_0 \prod_{i=1}^{\kappa} \mathbf{k}_i^{c_i}\right)$. In this section, we show how the final step of the oblivious evaluation leaks enough of the server key to allow a full key recovery attack in polynomial time.

4.1 Attack setup: Modifying the rounding step

To attack Leap , a malicious client $\mathcal{U}$ will follow the protocol described in [HKL+25] faithfully until the rounding step:

1. $\mathcal{U}$ fixes an input $\mathbf{c}$ and performs the Subset-Sum phase of the protocol honestly, and gets κ outputs $[\mathbf{t}_1, \ldots, \mathbf{t}_\kappa]$.
2. $\mathcal{U}$ sums the results from the Subset-Sum step up and lifts the result to the Subset-Product representation $\mathbf{b} = \mathsf{lookup}\left(\sum_{i=1}^{\kappa} \mathbf{t}_i\right)$.
3. After the OLE step, $\mathcal{U}$ holds $\mathbf{w} = \mathbf{a} \cdot \mathbf{b} + \bar{\mathbf{e}}$. An honest user would input $\mathbf{y} = \mathsf{iNTT}(\mathbf{w})$ into the rounding protocol. However, $\mathcal{U}$ uses $\mathbf{y}' = \mathbf{y} + \mathbf{\Delta y}$ for some $\mathbf{\Delta y}$ as an input to the $\binom{q}{1}$-OT in the rounding step.
4. After the rounding computations are finished, $\mathcal{U}$ receives either $\lfloor \mathbf{y}' - \mathbf{e} \rceil_2$ for Leap-CRT, or, for Leap-BCH, $(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2 + \mathbf{d}, \mathsf{BCH}(\mathbf{d}))$, where $\mathbf{d}$ is unknown to $\mathcal{U}$. In both cases, $\mathbf{e} = \mathsf{iNTT}(\bar{\mathbf{e}})$.

4.2 Attacking the CRT mode of Leap

We start with the simpler attack on the CRT mode and show how to fully recover the key in $(\kappa+1)\log \bar{q}$ OPRF protocol executions. After the iNTT transformation, the client holds $\mathbf{y} = \mathbf{ab} + \mathbf{e}$. A malicious client can now recover $\mathbf{ab}$ in at most $\log \bar{q}$ tries using a binary search algorithm by varying $\mathbf{y}$, since $\mathbf{ab}$ is constant and equal to $\mathbf{k}_0 \prod_{i=1}^{\kappa} \mathbf{k}_i^{c_i}$. In Figure 4, we explain how to recover $\mathbf{k}_0$ in $\log \bar{q}$ executions of Leap. To recover the remaining keys, the attacker repeats the same procedure, but sets the ith bit of $\mathbf{c}$ to recover the ith key.

1. On every iteration of the protocol, the client queries the OPRF result with the zero vector as a client input $\mathbf{c} = [0]^\kappa$.
2. On the first iteration, the client queries the honest OPRF result $\mathbf{v}_0 = \lfloor \mathbf{k}_0 \rceil_2$. The bit $v_{0,i}$ is set if and only if $\frac{\bar{q}}{4} \leq k_{0,i} < \frac{3\bar{q}}{4}$. In both cases, $k_{0,i}$ belongs to an interval $[s_i, t_i]$ of length $\frac{\bar{q}}{2}$.
3. On the second iteration of the protocol, the client sets $\mathbf{\Delta y} = \left(\ldots, \lfloor \frac{s_i+t_i}{2} - \frac{\bar{q}}{4} \rfloor, \ldots\right)$ and inputs $(\mathbf{y}+\mathbf{\Delta y})$ into the $\binom{\bar{q}}{1}$-OT. The client obtains the OPRF result $\mathbf{v}_1 = \lfloor \mathbf{k}_0 + \mathbf{\Delta y} \rceil_2$. Now the bit $v_{1,i}$ is set if and only if $k_{0,i}$ belongs to the second half of $[s_i, t_i]$. In both cases, the client determines that $k_{0,i}$ belongs to an interval of length at most $\lceil \frac{\bar{q}}{4} \rceil$.
4. The malicious adversary continues to update $\mathbf{\Delta y}$, as described in step 3, for $\lceil \log \bar{q} \rceil - 2$ more steps. After every iteration, the length of the interval containing $k_{0,i}$ is halved for every i.
5. In the final step, every entry of $\mathbf{k}_0$ belongs to an interval with only one element, so the key is recovered by the client.

Fig. 4: Procedure to recover a key in Leap-CRT. To recover the ith key, the ith bit of $\mathbf{c}$ has to be set in the first step.

Complexity. A full break requires $(\kappa + 1) \log \bar{q}$ protocol execution, as each recovery of the $\kappa + 1$ keys requires $\log \bar{q}$ searches. Since the CRT version of Leap is completely linear in the rounding step, we can parallelize the search for each coefficient at once.

4.3 Attacking the BCH mode of Leap.

The BCH mode of Leap encodes the rounding results with the BCH extension code. More concretely, the client obtains the two values

$$(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2 + \mathbf{d}, \mathsf{BCH}(\mathbf{d}))$$

after calling the $\binom{q}{1}$-OT in the rounding step. Subtracting $\mathsf{BCH}(\mathbf{d})$ from $\mathsf{BCH}(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2 + \mathbf{d})$ results in the evaluation $\mathsf{BCH}(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2)$. As a reminder, expanding the terms as in Equation (1) gives the correct NR-PRF evaluation $\mathbf{K}^\mathbf{c} = \mathsf{iNTT}(\mathbf{k}_0 \cdot \prod_{i=1}^{\kappa} \mathbf{k}_i^{c_i})$ of the input $\mathbf{c}$ chosen in the first step of the attack.

$$\begin{aligned} \mathsf{BCH}(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2 + \mathbf{d}) - \mathsf{BCH}(\mathbf{d}) &= \mathsf{BCH}(\lfloor \mathbf{y}' - \mathbf{e} \rceil_2) \\ &= \mathsf{BCH}(\lfloor \mathbf{\Delta y} + \mathbf{y} - \mathbf{e} \rceil_2) \\ &= \mathsf{BCH}(\lfloor \mathbf{\Delta y} + \mathsf{iNTT}(\mathbf{a} \cdot \mathbf{b}) \rceil_2) \\ &= \mathsf{BCH}(\lfloor \mathbf{\Delta y} + \mathbf{K}^\mathbf{c} \rceil_2) \end{aligned} \tag{1}$$

The main challenge when translating the attack on the CRT mode to the BCH mode is that the additional, linear application of the BCH code spreads

any local modification of the input to the rounding step to the entire output. As a result, we will only be able to recover one coefficient at a time, in contrast to Leap-CRT, where we can recover all coefficients of a key simultaneously. To be more precise, attacking the BCH mode increases the attack complexity by a factor of n in comparison to the CRT attack.

Setup. To recover the full key $\mathbf{K}$, we again start with the recovery of $\mathbf{k}_0$. After fully recovering $\mathbf{k}_0$, we can recover the remaining keys similarly. To simplify the attack, we will simplify the final step as a challenge-response protocol. The adversarial client sends $\mathbf{\Delta y}$ to the challenging server, who in the game responds with $\mathsf{BCH}(\lfloor \mathbf{\Delta y} + \mathbf{x} \rceil_2)$. The adversary aims to recover $\mathbf{x}$ in polynomial time, which is equivalent to recovering $\mathbf{k}_0$ from $\mathsf{BCH}(\lfloor \mathbf{k}_0 \rceil_2)$. Since the BCH code spreads any changes of a single polynomial coefficient to the entire output, we can only recover one coefficient at a time. We start with the first coefficient x_0, and then iteratively recover all other coefficients.

Key recovery. The first step of key recovery determines the offset t where the rounded value of x_0 changes. The initial query of the adversary carries out the protocol faithfully, with $\mathbf{\Delta y} = \mathbf{0}$ to get $\mathsf{BCH}(\lfloor \mathbf{x} \rceil_2)$.

To recover t, the adversary again performs dichotomic search as in the Leap-CRT attack Section 4.2, but with an integer instead of a full vector. Now, we adapt the value $a \in \mathbb{Z}_q$, where the adversary sends $\mathbf{\Delta y} = (a, 0, \ldots, 0)$. The challenger responds with

$$\mathsf{BCH}(\lfloor a\mathbf{e_1} + \mathbf{x} \rceil_2) = \mathsf{BCH}(\lfloor x_0 + a \rceil_2, \lfloor x_1 \rceil_2, \ldots, \lfloor x_{n-1} \rceil_2) = \mathbf{s}'$$

. There are two possible values for $\mathbf{s}'$:

$$\mathbf{s}' = \begin{cases} \mathbf{s}, & \lfloor x_0 + a \rceil_2 = \lfloor x_0 \rceil_2 \\ \mathbf{s} + \mathsf{BCH}(\mathbf{e}_1), & \lfloor x_0 + a \rceil_2 \neq \lfloor x_0 \rceil_2 \end{cases} \tag{2}$$

For the concrete parameters of Leap-BCH, the rounding result is 0 when $x_0 + a \in \{193, \ldots, 256\}, \{0, 192\}$, and 1 if $x_0 + a \in \{65, \ldots, 192\}$. Consequently, the rounding result of 128 consecutive values of a will be 1, and 0 for the remaining 129 results. The dichotomic search now aims to find the value t where $\mathbf{s}'(t) = \mathbf{s}$ and $\mathbf{s}'(t+1) = \mathbf{s} + \mathsf{BCH}(\mathbf{e}_1)$. Since the rounding function changes twice (between 64 and 65 and 192 and 193), we reduced the possible values of x_0 to $x_0 = 64 - t \vee x_0 = 192 - t$.

To check which value is the correct one, we use the *bias* introduced by the rounding function and call the oracle again with $\mathbf{\Delta y} = (t + 129, 0, \ldots, 0)$. If the challenger returns $\mathbf{s} + \mathsf{BCH}(\mathbf{e}_1)$, this indicates that $\mathbf{s}'$ is constant for 129 values. The rounding output is constant for 129 consecutive values when it is equal to 0, so $x_0 = 192 - t$. Otherwise, $x_0 = 64 - t$.

Recovering x_0 requires $\log q$ OPRF calls. To recover the remaining $n - 1$ values, we need to perform $n - 1$ additional dichotomic searches with $a\mathbf{e}_2, \ldots, a\mathbf{e}_n$.

Complexity. Recovering a key in Leap-BCH is more complex than for Leap-CRT, as recovering a single coefficient of a key requires $n \log q$ protocol executions. Since the attack is logarithmic in q, increasing the modulus does not effectivelly mitigate the attack.

5 Mitigating the attack on Leap

The full break of Leap can be mitigated by forcing the client to behave honestly or fixing the rounding step in Leap.

5.1 Removing the bias from Leap-BCH

The first mitigation removes the leakage from the rounded inputs of the $\binom{q}{1}$-OT, which mitigates the leakage sufficiently for Leap-BCH.

The rounding bias in Leap-BCH allows us to recover the exact value of the keys. However, we observe that removing the rounding bias reduces the probability of correctly guessing a coefficient to $\frac{1}{2}$. The adversary can verify a polynomial guess correct with a probability of $\frac{1}{2}^n$, which is sufficient for our security guarantees.

A standard technique in cryptography to mitigate the bias is masking. We observe that we only need to add a single unbiased element to the biased subset-product to remove the bias. This unbiased element could be the key polynomial $\mathbf{k}_0$, as it is not revealed to the client during the subset-sum computation which gets its efficiency from the uneven modulus. In contrast, $\mathbf{k}_0$ can be added later in the computation.

Initially, we wanted to add $\mathbf{k}_0$ just before the rounding step. There, the client holds the OLE output $\mathbf{y}$ and the server holds the unblinding polynomial $\mathbf{e}$, such that $\mathbf{y} - \mathbf{e} = \mathbf{k}_0 \prod_{i=1}^{\kappa} \mathbf{k}_i^{x_i}$. In the rounding step, the server computes $\lfloor y_i - e_i \rceil$ for all q possible polynomial coefficients y_i and inputs them into the $\binom{q}{1}$-OT. Adding $\mathbf{k}_0$ into the terms is tricky, as we left the NTT domain and cannot perform coefficient-wise operations.

Instead, we multiply $\mathbf{k}_0$ on the server side *after* the sum of unblinding terms has been lifted to the subset-product representation. This lightweight change adds 156[2] cycles to the server-side computation.

Choice of even modulus for $\mathbf{k_0}$ Natural choices for the modulus q are 514, as in Spring-CRT, 258, a bit more than the original q, or 256, a bit less. As there are no known attacks against lattices with $q = 256$, and the lattice estmator does not report a problem with this modulus, we chose 256 as it is the least invasive change to Leap. As outlined in the preliminaries Section 2.1, the polynomials

[2] The reference implementation absorbs $\mathbf{k}_0$ in four `_mm_256_add_epi8` instructions, each at one cycle. The multiplication takes five cycles each for 32 calls of `_mm256_mul_epi32`.

need to be invertible to avoid a trivial attack. Therefore, we have to sample from $\mathbb{Z}_{256}^*$.

The mitigation also works when all polynomials have an even modulus, i.e. Leap-CRT, at the expense of adding the BCH code every time.

5.2 Removing the structure from Leap-CRT

The attack on the CRT mode from Section 4.2 uses the linear relation between the blinded OLE output and the rounded unblinding value to recover the server key. The same linear relation is removed by the addition of the BCH code Leap-BCH, which makes the attack from Section 4.3 more complex. The key fully recovered since the uneven modulus introduces a bias. The bias leaks the exact value of the rounding results, together with the linear search.

Reducing the attack surface for $\mathbf{k_0}$. Intuitively, a third mode an even modulus, but an application of the BCH mode, would mitigate the full recovery: the malicious client could still recover t and reduce the possible values of each coefficient of e.g. the first coefficient x_0 of $\mathbf{k}_0$ to $x_0 = 64 - t \vee x_0 = 192 - t$. However, since the intervals are of the same size, the client can only reduce the key space from q^n to 2^n. While this is a significant reduction, the complexity of correctly guessing $\mathbf{k}_0$ is $(1/2)^n$. Since the polynomial dimension $n = 128$, this is still practically secure.

Recovering the remaining keys K. We observe that after recovering $\mathbf{k}_0$, Leap only exposes the remaining keys $\mathbf{k}_1, \ldots, \mathbf{k}_\kappa$ via *polynomial products* with $\mathbf{k}_0$. While the coefficients of $\mathbf{k}_0\mathbf{k}_i$ can still be determined with a probability of $(1/2)^n$, recovering $\mathbf{k}_1$ harder because of the multiplication with the partially unknown $\mathbf{k}_0$ preserves sufficient entropy to conceal $\mathbf{k}_1$.

Notation. Each coefficient of $\mathbf{k}_0$ is determined with a probability of 1/2. Write the two possible coefficients for the first coefficient x_0 as $(x_{0,0} \vee x_{0,1})$. Using this notation, $\mathbf{k}_0$ can be rewritten as $((x_{0,0} \vee x_{0,1}), (x_{1,0} \vee x_{1,1}), \ldots, (x_{n-1,0} \vee x_{n-1,1}))$.

Example 1 (Recovering a single coefficient y_0 of $\mathbf{k}_1$). To recover y_0 of the polynomial

$$\mathbf{k}_1 = (y_0, \ldots, y_{n-1})$$

, consider the first coefficient z_0 of $\mathbf{k}_0\mathbf{k}_1$. It has two possible values: $z_{0,0}$ and $z_{0,1}$. From the polynomial product, we obtain

$$(z_{0,0} \vee z_{0,1}) = (y_0x_{0,0} \vee y_0x_{0,1}) + (y_0x_{1,0} \vee y_0x_{1,1}) + \ldots + (y_0x_{n-1,0} \vee y_0x_{n-1,1})$$

. This can be rewritten as: $((192, 64) - z_0) = (y_0((192, 64) - x_0)) + (y_0((192, 64) - x_1)) + \ldots + (y_0((192, 64) - x_{n-1}))$. However, the multiplication obscures the concrete strucutre, since $y_0((192, 64) - x_0) \mapsto ((192y_0, 64y_0) - x_0y_0)$, which cannot be distinguished.

5.3 Forcing the client to behave: client consistency proofs

A generic method for preventing the client-side attack described in Section 4 is to require the client to provide a cryptographic proof of honest protocol execution. Any deviation from the protocol would result in a proof that fails to verify, allowing the server to abort the protocol.

The attack succeeds by having the client provide a maliciously modified input to the final protocol step. To mitigate this attack, the client has to prove that the input to the $\binom{q}{1}$-OT is the $\mathsf{iNTT}$of the output received from the OLE. To achieve security against a fully malicious client who may deviate at any step of the protocol, the client would be required to commit to the input $\mathbf{c}$, and prove consistency between inputting $\mathbf{c}$ into the $\binom{2}{1}$-OT for the subset-sum, prove that the input to the OLE $\mathbf{b}$ is consistent with $\mathbf{b} = \prod_{i=1}^{\kappa} \mathbf{r}_i \mathbf{k}_i^{c_i}$, and that the input to the $\binom{q}{1}$-OT is consistent with inverse NTT of the OLE output $\mathbf{y}$. This consistency proof could be instantiated with general-purpose zero-knowledge proofs like STARKs. However, integrating the proofs would impose a substantial computation and communication overhead on Leap, countering the primary goal of being more efficient than similar protocols with zero-knowledge proofs. Therefore, we propose a more lightweight construction that removes the algebraic structure that enabled the attack.

6 A full introduction of Leap-CB

The attacks from Section 4 and the mitigations from Section 5 hint that an effective mitigation for the attacks requires the modulus to be even (like in Spring-CRT) with an additional finalization step where a code is applied to remove algebraic structure (as in Spring-BCH).

We observe in Section 4 that the key coefficients can be determined to be either $k_{i,j}$ or $k_{i,j} + \frac{q}{2}$ in the rounding step. As discussed in the same section, this information leakage still does not enable a full key recovery by the adversary. This observation motivates the consideration of a smaller modulus q to improve efficiency and reduce the size of the Naor-Reingold key. The original choice of $q = 257$ is motivated by the discrete logarithm notation, as a polynomial coefficient fits conveniently into one byte. Motivated by the rounding step leakage, we explore the parameter space further. Decreasing q decreases the noise-to-error ratio, which is the factor of how large the LWE error is relative to the modulus. For Leap and Spring , it is easy to analyze the noise-to-error as the distribution is uniform. When reducing q it is important to ensure that the key polynomials are units. As in the original Leap paper, we use the lattice estimator to assess the security of a smaller q.

We start with a very short overview of common attacks on lattice problems. Then, we discuss two new constructions for Leap: One where $\mathbf{k}_0$ removes the bias, and one where we combine the CRT and the BCH mode.

Brute force attack. In a brute force attack, the adversary tries all possible inputs or keys until the correct one is found. A sufficiently large search space

makes brute force attacks impractical. For Spring and Leap, a brute force attack searches for a secret $\mathbf{k}_i \in (R_q^*)^n$ among the rounded results $S(\mathbf{k}_0 \prod \mathbf{k}_i^{c_i})$ A brute force attack to recover a key takes $\frac{q}{2}^n$, which is prohibitively large for all $q \geq 3$. A brute force attack enables the attacks against Leap in Section 4.

Combinatorial Attacks. Combinatorial attacks exploit the generalized birthday problem on noisy learning problems. The attack first draws a large number of samples and then identifies small subsets whose sum lies within a sufficiently small subgroup using birthday collisions. If the subgroup can be used to detect noise, the adversary successfully constructed a distinguisher. A combinatorial attack that cancels out the R_2 component for Spring-CRT is described in Section 2.1.

Lattice Attacks. Lattice attacks query a large number of lattice samples to uniquely determine the secret with a good probability. We can in general recover the noisy key $\mathbf{k}_0' = \mathbf{k}_0 + \frac{q}{2}\mathbf{s}$. Clearly, the noise vector $\mathbf{s}$ is a uniformly random when $\mathbf{k}_0$ is uniformly random. This mitigates attacks against known structure or sparse vectors like Salsa [WCCL22, LSW+23].

Algebraic Attacks. The lattice attack that becomes feasible when reducing q is the linearization attack of Arora and Ge [AG11]. When the error terms are sampled from a narrow distribution, the adversary can distinguish the error terms belonging to a known set of size d. With enough samples, the attack recovers the LWE secret in polynomial time. When we decrease q, we also decrease the set size d. As visible in Table 1, linearization attacks are feasible for very small $q \leq 16$.

6.1 Leap-CB: CRT with BCH codes.

The attack against Spring-CRT shown in Section 5 leverages the linear relation between the OLE output and the rounding result to recover the key. On the other hand, the linear relation is not present in Leap-BCH. Instead the attack leverages the bias introduced by the uneven modulus to recover the key.

To mitigate the attacks, our key idea is to combine the CRT mode with an additional BCH code to remove the linear dependency. The resulting OPRF is effectively Figure 3, with an additional message of $\mathsf{BCH}(\mathbf{d})$. Even with the mitigations, an adversarial client can recover the noisy key $\mathbf{k}_0' \equiv \mathbf{k}_0 + \frac{q}{2}\mathbf{s}$, with a secret offset vector $\mathbf{s} \in \{0,1\}^n$. The probability of correctly guessing $\mathbf{s}$ is $\frac{1}{2}^{128}$, which is negligable and therefore sufficient to hide the actual key.

6.2 Reducing q.

Given that the new construction leaks almost the entire key, except the most significant bit, we now investigate whether the modulus q can be safely reduced.

Table 1 presents the hardness estimates for a range of smaller moduli between four and six bits. The original SPRING paper chose $q = 257$ for performance reasons, since it enabled efficient rounding through overflows. However, in the context of OPRFs, smaller primes are preferable the main drawback of lattice-based OPRFs is their high communication cost.

In our analysis, we assume we always round to two, and that the modulus is even, which will then be composed into its prime factors as in Figure 3. Following the methodology of the LEAP paper, we call the lattice estimator [APS15, ACD+18] , Commit ID `5ba00f56dd1086c3a42b98fc596c64907adb96ff`, with `LWE.Parameters(n=128, q=q, Xe=NoiseDistribution.UniformMod((q//2)), Xs=NoiseDistribution.UniformMod(2))`, restricting the number of LWE samples to 2^{60} and 2^{90}. For all parameter settings, the USVP, BDD and BDD-hybrid output are inifinity again for all parameter inputs. Table 1 presents parameter regimes where all known lattice attacks require prohibitively high memory or computation ressources.

Choosing $q = 14$ appears to be an attractive option. With four-bit arithmetic, it would cut the communication requirements of LEAP in half. Although this increases the complexity from an implementation perspective, such challenges are generally managable in practice. A limitation of the analysis with the lattice estimator is that parameters that small generally have not been studied. In LWR, the key security factor is the distance between the modulus and the rounding result, which requires further investigation. The alternative $q = 16$ could also be interesting, as the implementation would combine four fast two-arithmetic constructions.

6.3 UC proof for LEAP-CB

With a minimal modification of the proof of UC-security of LEAP-BCH, we can prove that LEAP-CB UC-realizes the functionality $\mathcal{F}_{\text{OPRF}}$ from [JKX18] that models an OPRF protocol with adaptive compromise, when both parties $\mathcal{S}$ and $\mathcal{U}$ are semihonest. Also in this case, LEAP-CB uses the functionalities $\mathcal{F}_{\binom{q}{1}\text{-OT}}$ from [ABB+13] and $\mathcal{F}_{\text{OLE}}$ from [GNN17] to perform the necessary $\binom{q}{1}$-OT and OLE protocols, respectively. Finally, we need to extend the final LEAP-CB output with a hash function H that will be modelled as a random oracle. The evaluation of the PRF on input $\mathbf{c}$ with keys $\mathbf{K} = (\mathbf{k}_0, \ldots, \mathbf{k}_\kappa)$ will be equal to

$$\mathcal{F}'_{\mathbf{K}}(\mathbf{c}) = H\left(\mathbf{c}, \mathcal{F}_{\mathbf{K}}(\mathbf{c})\right) = H\left(\mathbf{c}, \mathsf{BCH}\left(\left\lfloor \mathbf{k}_0 \prod_{i=1}^{\kappa} \mathbf{k}_i^{c_i} \right\rceil_2\right)\right).$$

In Figure 5, we describe LEAP-CB protocol within the UC framework.

Components: Hash function $H : \{0,1\}^* \times \{0,1\}^k \to \{0,1\}^\ell$, where ℓ polynomial in the security parameter. H is specific to the OPRF instance initialized for a unique session ID *sid*, and they should be implemented

by folding sid into their inputs.

Initialization: On (INIT, sid) from $\mathcal{Z}$, $\mathcal{S}$ picks $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa \leftarrow\$ R_{2q}$ and stores $(sid, (\mathbf{k}_0, \ldots, \mathbf{k}_\kappa))$. All elements $\mathbf{a} \in R_{2q}$ are sampled as $(\mathbf{a}_q, \mathbf{a}_2) \in \mathbb{Z}_{q-1}^n \times R_2$.

Server compromise: On (COMPROMISE, $sid, \mathcal{S}$) from $\mathcal{A}$, $\mathcal{S}$ reveals $(\mathbf{k}_0, \ldots, \mathbf{k}_\kappa)$.

Offline evaluation: On (OFFLINEEVAL, $sid, \mathcal{S}, \mathbf{x}$) from $\mathcal{Z}$, $\mathcal{S}$ recovers $(sid, (\mathbf{k}_0, \ldots, \mathbf{k}_\kappa))$ and returns (OFFLINEEVAL,$sid, \mathcal{F}_{\mathbf{K}}(\mathbf{x})$).

User online evaluation:
- On (EVAL, $sid, ssid, \mathcal{S}', \mathbf{x}$) from $\mathcal{Z}$, $\mathcal{U}$ stores $(sid, ssid, \mathbf{x})$; sends (STARTPROTOCOL, $sid, ssid$) to $\mathcal{S}'$, (PREFIX,$sid, ssid, ssid$) to $\mathcal{Z}$ and for $i \in [\kappa]$ sends (RECEIVE,$sid, (ssid, i), \mathcal{S}', \mathcal{U}, x_i$) to $\mathcal{F}_{\binom{2}{1}\text{-OT}}$.
- On (RECEIVED, $sid, (ssid, i), \mathcal{S}, \mathcal{U}, (\mathbf{t}_{i,q}, \mathbf{t}_{i,2})$) from $\mathcal{F}_{\binom{2}{1}\text{-OT}}$ for $i \in [\kappa]$, $\mathcal{U}$ evaluates $\mathbf{b}_q = \text{lookup}(\sum_{i=1}^{\kappa} \mathbf{t}_{i,q})$ and $\mathbf{b}_2 = \text{lookup}(\sum_{i=1}^{\kappa} \mathbf{t}_{i,2})$. For $j \in \{0, \ldots, n-1\}$ and $h \in \{2, q\}$, $\mathcal{U}$ sends (INPUTR, $sid, (ssid, (h, j))$, $\mathcal{S}, \mathcal{U}, b_{h,j}$) to $\mathcal{F}_{\text{OLE}}$.
- On (OUTPUT,$sid, (ssid, (h, i)), \mathcal{S}, \mathcal{U}, w_{h,i}$) from $\mathcal{F}_{\text{OLE}}$ for $i \in \{0, \ldots, n-1\}$ and $h \in \{2, q\}$, $\mathcal{U}$ stores $\mathbf{y}_q = \mathsf{iNTT}(\sum_{i=0}^{n-1} w_{q,i} X^i)$ and $\mathbf{y}_2 = \mathsf{iNTT}(\sum_{i=0}^{n-1} w_{2,i} X^i)$. From $\mathbf{y}_q$ and $\mathbf{y}_2$, $\mathcal{U}$ recovers $\mathbf{y} \in R_{2q}$. For $j \in [n]$, $\mathcal{U}$ sends (RECEIVE,$sid, (ssid, j), \mathcal{S}, \mathcal{U}, y_{j-1}$) to $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$.
- On (BCH, $ssid, \mathbf{s}$) from $\mathcal{S}$ and (RECEIVED,$sid, (ssid, i), \mathcal{S}, \mathcal{U}, u_i$) from $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$ for $i \in \{0, \ldots, n-1\}$; $\mathcal{U}$ sends (EVAL, $sid, ssid, H(x, \text{BCH}(\mathbf{u}) - \mathbf{s})$).

Server online evaluation: On (STARTPROTOCOL, $sid, ssid$) from $\mathcal{U}$ and (SNDRCOMPLETE, $sid, ssid'$) from $\mathcal{Z}$, $\mathcal{S}$ recovers $(sid, (\mathbf{k}_0, \ldots, \mathbf{k}_\kappa))$ and performs the following actions:

- $\mathcal{S}$ sends (PREFIX, $sid, ssid', ssid$) to $\mathcal{Z}$
- $\mathcal{S}$ samples $\mathbf{r}_0, \ldots, \mathbf{r}_\kappa \leftarrow\$ R_{2q}$
- for $i \in [\kappa]$ $\mathcal{S}$ sends (SEND,$sid, (ssid, i), \mathcal{S}, \mathcal{U}, (\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i)$) to $\mathcal{F}_{\binom{2}{1}\text{-OT}}$ and stores $\mathbf{a}_q = \text{lookup}(\mathbf{k}_{0,q} - \sum_{i=1}^{\kappa} \mathbf{r}_{i,q}), \mathbf{a}_2 = \text{lookup}(\mathbf{k}_{0,2} - \sum_{i=1}^{\kappa} \mathbf{r}_{i,2})$
- for $i \in \{0, \ldots, n-1\}$ and $h \in \{2, q\}$, $\mathcal{S}$ samples $\bar{e}_{h,i} \leftarrow\$ \mathbb{Z}_h$ and sends (INPUTS,$sid, (ssid, (h, i)), \mathcal{S}, \mathcal{U}, (a_{(h,i)}, \bar{e}_{(h,i)})$) to $\mathcal{F}_{\text{OLE}}$. $\mathcal{S}$ stores $\mathbf{e}_h = \mathsf{iNTT}(\sum_{i=0}^{n-1} \bar{e}_{(h,i)} X^i)$ for $h \in \{2, q\}$
- $\mathcal{S}$ recovers $\mathbf{e}$ from $\mathbf{e}_q$ and $\mathbf{e}_2$
- $\mathcal{S}$ samples $\mathbf{d} \leftarrow\$ \{0,1\}^n$ and for $i \in [n]$, $\mathcal{S}$ sends (SEND,$sid, (ssid, i), \mathcal{S}, \mathcal{U}, (\lfloor z - e_{i-1} \rceil_2 + d_i.\ z \in \mathbb{Z}_{2q})$) to $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$
- $\mathcal{S}$ sends (BCH, $ssid$, BCH($\mathbf{d}$)) to $\mathcal{U}$.

Fig. 5: LEAP-CB protocol in UC $\mathcal{F}_{\binom{N}{1}\text{-OT}}, \mathcal{F}_{\text{OLE}}-$hybrid world.

For every session ID sid, which is uniquely bound to a key $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$, **SIM** keeps a list $L_{sid} = \langle(\Delta, \mathcal{S}_\Delta)\rangle$ where $\Delta \in \{0,1\}^k$ and $\mathcal{S}_\Delta$ is a fake sender identity.

Initialization: On (INIT, $\mathcal{S}, sid$) from $\mathcal{F}$, **SIM** picks $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa \leftarrow\$ R_{2q}$ and stores $(\mathcal{S}, sid, (\mathbf{k}_0, \ldots, \mathbf{k}_\kappa))$.

Server compromise: On (COMPROMISE, $sid, \mathcal{S}$) from $\mathcal{A}$ meant for $\mathcal{S}$, **SIM** forwards it to $\mathcal{F}$ and reveals $(\mathbf{k}_0, \ldots, \mathbf{k}_\kappa)$.

H queries response: On (H QUERY, $\mathbf{x}, f, sid$) from $\mathcal{Z}$ meant for the oracle, **SIM** recovers $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$ associated to sid and evaluates $\Delta = f - \mathcal{F}_{\mathbf{K}}(x)$:
- if $\Delta = 0$ and $\mathcal{S}$ is not compromised **SIM** sends (HALT) to $\mathcal{Z}$
- if $\Delta = 0$ and $\mathcal{S}$ is compromised, **SIM** sends (OFFLINEEVAL, $sid, S, \mathbf{x}$) to $\mathcal{F}$ and shares its response $F_{sid,\mathcal{S}}(\mathbf{x})$ to $\mathcal{Z}$
- if $\Delta \neq 0$, **SIM** looks for Δ in L_{sid}. If it is not present, **SIM** picks a fresh identity $\mathcal{S}_\Delta$ and add $(\Delta, \mathcal{S}_\Delta)$ to L_{sid}. In both cases sends (OFFLINEEVAL, $sid, \mathcal{S}_\Delta, \mathbf{x}$) to $\mathcal{F}$ and shares its response $F_{sid,\mathcal{S}_\Delta}(\mathbf{x})$ to $\mathcal{Z}$.

User online evaluation:

- On (EVAL, $sid, ssid, \mathcal{U}, \mathcal{S}'$) from $\mathcal{F}$, **SIM** sends ($ssid$) to $\mathcal{F}$ as $\mathcal{U}$'s prefix. Then **SIM** shows $\mathcal{A}$ (STARTPROTOCOL, $ssid$) as a message from $\mathcal{U}$ to $\mathcal{S}'$ and (RECEIVE, $sid, (ssid, i), \mathcal{S}', \mathcal{U}$) for $i \in [\kappa]$ as messages from $\mathcal{F}_{\binom{2}{1}\text{-OT}}$
- On (RECEIVED, $sid, (ssid, i), \mathcal{S}, \mathcal{U}$) from $\mathcal{A}$ meant for $\mathcal{F}_{\binom{2}{1}\text{-OT}}$ for $i \in [\kappa]$ and only if $\mathcal{A}$ didn't change $ssid$ in the STARTPROTOCOL message, **SIM** shows $\mathcal{A}$ (INPUT, $sid, (ssid, (h, j))$) for $j \in \{0, \ldots, n-1\}, h \in \{2, q\}$ as messages from $\mathcal{F}_{\text{OLE}}$
- On (DELIVER, $sid, (ssid, (h, i)), \mathcal{S}, \mathcal{U}$) from $\mathcal{A}$ meant for $\mathcal{F}_{\text{OLE}}$ for $i \in \{0, \ldots, n-1\}, h \in \{2, q\}$, **SIM** shows $\mathcal{A}$ (RECEIVE, $sid, (ssid, j), \mathcal{S}, \mathcal{U}$) for $j \in [n]$ as messages from $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$
- On (RECEIVED, $sid, (ssid, i), \mathcal{S}, \mathcal{U}$) from $\mathcal{A}$ meant for $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$ for $i \in [n]$ and on (BCH, $ssid, \mathbf{s}'$) from $\mathcal{A}$ as a message from $\mathcal{S}$ to $\mathcal{U}$, **SIM** retrieves the stored syndrome $\mathbf{s}$ and $\mathcal{A}$ modified and evlauates $\Delta = \mathbf{s}' - \mathbf{s}$:
  - if $\Delta = 0$ **SIM** sends (RCVCOMPLETE, $sid, ssid, \mathcal{U}, \mathcal{S}$) to $\mathcal{F}$
  - if $\Delta \neq 0$, **SIM** looks for Δ in L_{sid}. If it is not present,**SIM** picks a fresh identity $\mathcal{S}_\Delta$ and add $(\Delta, \mathcal{S}_\Delta)$ to L_{sid}. In both cases sends (RCVCOMPLETE, $sid, ssid, \mathcal{U}, \mathcal{S}_\Delta$) to $\mathcal{F}$

Server online evaluation: On (STARTPROTOCOL, $sid, ssid'$) from $\mathcal{A}$ as a message from $\mathcal{U}$ to $\mathcal{S}$ and (SNDRCOMPLETE, $sid, ssid, \mathcal{S}$) from $\mathcal{F}$, **SIM** performs the following actions:

- **SIM** sends ($ssid'$) to $\mathcal{F}$ as $\mathcal{S}$'s prefix
- for $i \in [\kappa]$ **SIM** sends (SEND,$sid, (ssid', i), \mathcal{S}, \mathcal{U}$) to $\mathcal{A}$ as a message from $\mathcal{F}_{\binom{2}{1}\text{-OT}}$.
- for $i \in [n], h \in \{2, q\}$, **SIM** sends (INPUT,$sid, (ssid', (h, i-1)), \mathcal{S}, \mathcal{U}$) to $\mathcal{A}$ as a message from $\mathcal{F}_{\text{OLE}}$.
- for $i \in [n]$, **SIM** sends (SEND,$sid, (ssid', i), \mathcal{S}, \mathcal{U}$) to $\mathcal{A}$ as a message from $\mathcal{F}_{\binom{2q}{1}\text{-OT}}$
- **SIM** samples $\mathbf{s} \leftarrow\$ \{0,1\}^k$, stores ($\mathbf{s}, ssid'$) and shows $\mathcal{A}$ (BCH, $ssid', \mathbf{s}$) as message from $\mathcal{S}$ to $\mathcal{U}$.

Fig. 6: **SIM** behavior in the ideal world.

Theorem 1. *The protocol in Figure 5 for $\mathcal{F}'_{\mathbf{K}}$ realizes the functionality $\mathcal{F}_{OPRF}$ if the OLE protocol realizes the functionality $\mathcal{F}_{OLE}$, and the two OT protocols realize the functionality $\mathcal{F}_{\binom{N}{1}\text{-}OT}$ in the ROM. More precisely, for every adversary $\mathcal{A}$ there exists a simulator* **SIM** *that produces a view in the ideal world that no environment $\mathcal{Z}$ can distinguish with advantage better than $\frac{h}{2^k}$ where h is the number of H query performed by $\mathcal{Z}$ on an uncompromised server.*

Proof. We can always assume that $\mathcal{A}$ acts as a dummy adversary who follows $\mathcal{Z}$'s instructions and shares its view to $\mathcal{Z}$. For every possible environment $\mathcal{Z}$ we will use the simulator **SIM** described in Figure 6.

First, let us consider the messages that **SIM** can craft and are trivially identical to the protocol execution in the hybrid world. These messages are completely independent of any piece of information that is involved in the protocol except the identity of the participants and the session IDs sid and $ssid$. The messages are:

- STARTPROTOCOL from $\mathcal{U}$ (if $\mathcal{A}$ changes this message the protocol cannot reach the end, otherwise the prefixes always match and the evaluation of the final output is always permitted)
- PREFIX from $\mathcal{S}$ and $\mathcal{U}$ (through communications with $\mathcal{F}$)
- SEND, RECEIVE from $\mathcal{F}_{\binom{N}{1}\text{-OT}}$
- INPUT from $\mathcal{F}_{\text{OLE}}$
- BCH from $\mathcal{S}$ to $\mathcal{U}$ since $\mathbf{d}$ has uniform distribution and BCH($\cdot$) is a linear application.

Now we analyze the messages that are not identical in the two world but indistinguishable to $\mathcal{Z}$.

Let us analyze the view of $\mathcal{Z}$ in the real world:

- On H QUERY on input $(\mathbf{x}, f, sid)$ it receives $H(\mathbf{x}, f)$.
- On COMPROMISE $\mathcal{S}$ from $\mathcal{A}$, $\mathcal{Z}$ recovers the secret keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$.
- On EVAL message to $\mathcal{U}$ on input $\mathbf{x}$, with sender $\mathcal{S}$ who has keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$, $\mathcal{Z}$ can ask $\mathcal{A}$ to modify the syndrome $\mathbf{s}$ to $\mathbf{s}' = \mathbf{s} + \Delta$ for some Δ. The output from $\mathcal{U}$ is
$$H\left(x, \mathcal{F}_{\mathbf{K}}(\mathbf{x}) + \Delta\right).$$
Note that if the keys are compromised then $\mathcal{Z}$ has full control over the second input of H.

What happens in the ideal world is:

- On H QUERY on input $(\mathbf{x}, f, sid)$, **SIM** recovers $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$ that it has associated to sid, evaluates
$$\Delta = f - \mathcal{F}_{\mathbf{K}}(x)$$
and if $\Delta \neq 0$, returns $F_{sid,\mathcal{S}_\Delta(\mathbf{x})}$ to $\mathcal{Z}$. This is always possible sending a OFFLINEEVAL message to $\mathcal{F}$ since $\mathcal{S} \neq \mathcal{S}_\Delta$. If $\Delta = 0$ and the keys are compromised then, thanks to OFFLINEEVAL **SIM** can obtain $F_{sid,\mathcal{S}}(\mathbf{x})$ and share it to $\mathcal{Z}$. If $\Delta = 0$ and the keys are not compromised, then **SIM** cannot produce the correct output and must HALT the simulation.
- On COMPROMISE $\mathcal{S}$ from $\mathcal{A}$, **SIM** discloses to $\mathcal{Z}$ the keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$ associated to $\mathcal{S}$.
- On EVAL message to $\mathcal{U}$ on input x, with sender $\mathcal{S}$, $\mathcal{Z}$ can ask $\mathcal{A}$ to modify the syndrome $\mathbf{s}$ to $\mathbf{s}' = \mathbf{s} + \Delta$ for some Δ. **SIM** can clearly retrieve Δ. If the syndrome is modified $\mathcal{Z}$ receives $F_{sid,\mathcal{S}_\Delta}(\mathbf{x})$, otherwise $F_{sid,\mathcal{S}}(\mathbf{x})$.

So far, **SIM** is able to produce an output in most cases. Multiple EVAL output with the same interaction from $\mathcal{Z}$ are consistent; and the same works for H queries. We have to verify that the output created with a H QUERY is coherent with what is obtained during the protocol.

So suppose that $\mathcal{S}$ has uncompromised keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$. Let us assume that $\mathcal{Z}$ performs multiples H queries on the same server $\mathcal{S}$ on the inputs $\{(\mathbf{x}_i, f_i)\}_i$ and start the protocol on inputs $\{\mathbf{x}_j\}_j$ and modifies the syndromes with $\{\Delta_j\}_j$. Let us define $\Delta_i = f_i - \mathcal{F}_{\mathbf{K}}(\mathbf{x}_i)$ and suppose for a moment that all of them are nonzero. We can also suppose that all Δ_j are different from 0, otherwise a correct $F_{sid,\mathcal{S}}(\mathbf{x}_j)$ is returned.

From the H queries $\mathcal{Z}$ receives $\{F_{sid,\mathcal{S}_{\Delta_i}}(\mathbf{x}_i)\}_i$, whereas from the protocol executions $\mathcal{Z}$ receives $\{F_{sid,\mathcal{S}_{\Delta_j}}(\mathbf{x}_j)\}_j$. Without knowing $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$, the second input of H in a protocol execution is unknown to $\mathcal{Z}$, so it cannot do anything with these results.

If $\mathcal{Z}$ decides to COMPROMISE $\mathcal{S}$, then it has access to the keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$:

- for every $\mathbf{x}_j$, $\mathcal{Z}$ can evaluate $f_j = \mathcal{F}_{\mathbf{K}}(\mathbf{x}_j) + \Delta_j$ and ask for a H query on input $(\mathbf{x}_j, f_j)$. At this point **SIM** will evaluate $\Delta = f_j - \mathcal{F}_{\mathbf{K}}(\mathbf{x}_j) = \Delta_j$. So the result returned is exactly $F_{sid,\mathcal{S}_{\Delta_j}}(\mathbf{x}_j)$.

– for every $(\mathbf{x}_i, f_i)$, $\mathcal{Z}$ can evaluate $\Delta_i^* = f_i - \mathcal{F}_{\mathbf{K}}(\mathbf{x}_i) = \Delta_i$. In order to obtain $H(\mathbf{x}_i, f_i)$ through the protocol, $\mathcal{Z}$ needs to modify the syndrome in $\mathbf{s}' = \mathbf{s} + \Delta_i$. **SIM**, following its procedure, will make $\mathcal{U}$ return $F_{sid,\mathcal{S}_{\Delta_i}}(\mathbf{x}_i)$ to $\mathcal{Z}$.

All messages are coherent and there is no way for $\mathcal{Z}$ to distinguish reality between simulation. If $\mathcal{Z}$ tries to evaluate H on new inputs after the server compromise, the situation does not change since **SIM** is now in a stronger position.

Let us now consider the probability that the HALT message is shared by the simulator. This event happens if and only if $\mathcal{Z}$ perform a query to the H oracle on input $(\mathbf{x}_i, f_i, sid)$ where sid is related to the uncompromised keys $\mathbf{k}_0, \ldots, \mathbf{k}_\kappa$ and $f_i = \mathcal{F}_{\mathbf{K}}(\mathbf{x}_i)$ (which is to say, $\Delta_i = 0$). Since both participants are honest, through protocol executions $\mathcal{Z}$ can only recover $\mathbf{x}_i$, $H(\mathbf{x}_i, \mathcal{F}_{\mathbf{K}}(\mathbf{x}_i))$ and the syndromes $\mathbf{s}$ which have uniform distribution (since $\mathbf{d}$ have). For this reason, it is impossible for $\mathcal{Z}$ to obtain information about $\mathcal{F}_{\mathbf{K}}(\mathbf{x}_i)$. The only thing that $\mathcal{Z}$ can do is guessing the correct output which happens with a probability of $1/2^k$ per query. The total advantage that $\mathcal{Z}$ has to distinguish the views is exactly $\frac{h}{2^k}$.

6.4 Leap-CB is a weak OPRF

The mitigation slightly weakens the original security claims since an adversary can learn partial information about the key. In an ideal OPRF, the client learns nothing about the key. However, this definition is *often too strong and it does not allow to prove security of many efficient OPRF protocols* [CHL22]. Freedman et al. [FIPR05] define a weak, or relaxed, OPRF as an OPRF where the client may learn *partial* information about the key or key-dependent variables, but not information that compromises pseudorandomness of yet unqueried inputs [FIPR05, Definition 6]. A weak OPRF may reveal *the first bit of the PRF key, or the preimage of the final key derivation function* [CHL22]. For example, the popular 2HashDH OPRF [JKK14] reveals the preimage $H(x)^k$ to the client, who then locally computes the final result $H'(x, H(x)^k)$. Similarily, Leap-CB leaks more information than original mode, but does not compromise the pseudorandomness of yet unqueried inputs and therefore still fits within the weak OPRF definition.

7 Conclusion

In this paper, we introduced two key recovery attacks on Leap in a malicious setting, targeting both the CRT and the BCH modes, and analyzed the generic Naor-Reingold OPRF in a malicious server setting. To faciliate these attacks, we provided a complete description of the Leap-CRT mode that was omitted in the orginal paper.

To mitigate the vulnerabilites, we proposed several countermeasures, including bias removal, structure removal and zero-knowledge proofs to deter malicous

behaviour. Building on these ideas, we introduced Leap-CB, a new mode for Leap that combines both the CRT and the BCH mode to reduce leakage. While some information about the server key still leaks, the new mode effectively mitigates the dedicated attacks. We then discuss how to leverage the leakage to get a more efficient protocol with smaller moduli.

A natural direction for future work is to investigate the security of the small moduli further, and write a full malicious security proof for the Leap-CB.

Acknowledgements. Anonymized

References

ABB+13. Michel Abdalla, Fabrice Benhamouda, Olivier Blazy, Céline Chevalier, and David Pointcheval. SPHF-friendly non-interactive commitments. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 214–234. Springer, Berlin, Heidelberg, December 2013.

ACD+18. Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE, NTRU schemes! In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 351–367. Springer, Cham, September 2018.

AG11. Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jiri Sgall, editors, *ICALP 2011, Part I*, volume 6755 of *LNCS*, pages 403–415. Springer, Berlin, Heidelberg, July 2011.

APS15. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.

BBL+15. Abhishek Banerjee, Hai Brenner, Gaëtan Leurent, Chris Peikert, and Alon Rosen. SPRING: Fast pseudorandom functions from rounded ring products. In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 38–57. Springer, Berlin, Heidelberg, March 2015.

BCG+19. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.

BDFK17. Charles Bouillaguet, Claire Delaplace, Pierre-Alain Fouque, and Paul Kirchner. Fast lattice-based encryption: Stretching spring. In Tanja Lange and Tsuyoshi Takagi, editors, *Post-Quantum Cryptography - 8th International Workshop, PQCrypto 2017*, pages 125–142. Springer, Cham, 2017.

Bea96. Donald Beaver. Correlated pseudorandomness and the complexity of private computations. In *28th ACM STOC*, pages 479–488. ACM Press, May 1996.

BFH+20. Carsten Baum, Tore Kasper Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. PESTO: Proactively secure distributed single sign-on, or how to trust a hacked server. In *2020 IEEE European Symposium on Security and Privacy*, pages 587–606. IEEE Computer Society Press, September 2020.

BPR12. Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, April 2012.

CDFH+22. S. Celi, A. Davidson, A. Faz-Hernandez, S. Valdes, and C. A. Wood. The privacy pass protocol. Internet Draft, April 2022.

CHL22. Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. Sok: Oblivious pseudorandom functions. In *7th IEEE European Symposium on Security and Privacy, EuroS&P 2022, Genoa, Italy, June 6-10, 2022*, pages 625–646. IEEE, 2022.

FIPR05. Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 303–324. Springer, Berlin, Heidelberg, February 2005.

GNN17. Satrajit Ghosh, Jesper Buus Nielsen, and Tobias Nilges. Maliciously secure oblivious linear function evaluation with constant overhead. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 629–659. Springer, Cham, December 2017.

HHM+24. Lena Heimberger, Tobias Hennerbichler, Fredrik Meisingseth, Sebastian Ramacher, and Christian Rechberger. OPRFs from isogenies: Designs and analysis. In Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas, editors, *ASIACCS 24*. ACM Press, July 2024.

HKL+25. Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Rechberger. Leap: A fast, lattice-based OPRF with application to private set intersection. Cryptology ePrint Archive, Paper 2025/333, 2025.

HL08. Carmit Hazay and Yehuda Lindell. Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 155–175. Springer, Berlin, Heidelberg, March 2008.

JKK14. Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.

JKR19. Stanislaw Jarecki, Hugo Krawczyk, and Jason K. Resch. Updatable oblivious key management for storage systems. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 379–393. ACM Press, November 2019.

JKX18. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.

KKRT16. Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious PRF with applications to private set intersection. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 818–829. ACM Press, October 2016.

KLÖ25. Maximilian Kroschewski, Anja Lehmann, and Cavit Özbay. OPPID: single sign-on with oblivious pairwise pseudonyms. *Proc. Priv. Enhancing Technol.*, 2025(2):629–649, 2025.

KRS+19. Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1447–1464. USENIX Association, August 2019.

LSW+23. Cathy Yuanchen Li, Jana Sotáková, Emily Wenger, Mohamed Malhou, Evrard Garcelon, François Charton, and Kristin E. Lauter. SalsaPicante: A machine learning attack on LWE with binary secrets. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 2606–2620. ACM Press, November 2023.

WCCL22. Emily Wenger, Mingjie Chen, Francois Charton, and Kristin E Lauter. Salsa: Attacking lattice cryptography with transformers. *Advances in Neural Information Processing Systems*, 35:34981–34994, 2022.

Table 1: Small-moduli parameter sets where known attacks are infeasible due to excessive computational complexity *rop* or memory requirements *mem*.

			arora-gb		bkw		dual		dual_hybrid		bdd_mitm_hybrid	
m	n	q	rop	mem	rop	mem	rop	mem	rop	mem	rop	mem
2^{60}	128	14	$2^{97.6}$	$2^{97.6}$	$2^{95.6}$	$2^{43.5}$	$2^{108.6}$	$2^{70.8}$	$2^{108.4}$	-	$2^{164.9}$	-
2^{60}	128	16	$2^{120.1}$	$2^{109.1}$	-	-	$2^{108.6}$	$2^{70.8}$	$2^{108.5}$	-	$2^{168.0}$	-
2^{60}	128	20	2^{inf}	-	-	-	$2^{108.7}$	$2^{71.2}$	$2^{108.5}$	-	$2^{174.2}$	-
2^{60}	128	24	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{179.1}$	-
2^{60}	128	28	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{183.3}$	-
2^{60}	128	32	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{187.8}$	-
2^{60}	128	36	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{191.3}$	-
2^{60}	256	10	$2^{91.2}$	$2^{76.2}$	$2^{312.5}$	$2^{30.9}$	$2^{169.8}$	$2^{112.2}$	$2^{204.1}$	-	-	-
2^{60}	256	12	$2^{126.2}$	$2^{126.2}$	-	-	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{60}	256	14	$2^{169.8}$	$2^{168.8}$	$2^{303.9}$	$2^{35.3}$	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{60}	256	16	$2^{213.0}$	$2^{206.0}$	-	-	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{60}	256	18	2^{inf}	-	$2^{265.7}$	$2^{60.0}$	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{60}	256	20	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{60}	256	22	2^{inf}	-	$2^{255.7}$	$2^{64.0}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{60}	256	24	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{60}	256	26	2^{inf}	-	$2^{238.1}$	$2^{67.4}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{60}	256	28	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{60}	256	30	2^{inf}	-	$2^{232.1}$	$2^{70.3}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	$2^{410.0}$	-
2^{60}	256	32	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{414.3}$	-
2^{60}	256	34	2^{inf}	-	$2^{167.5}$	$2^{87.5}$	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{417.6}$	-
2^{60}	256	36	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{421.7}$	-
2^{60}	256	38	2^{inf}	-	$2^{158.5}$	$2^{90.2}$	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{425.1}$	-
2^{90}	128	14	$2^{97.6}$	$2^{97.6}$	$2^{95.6}$	$2^{43.5}$	$2^{108.6}$	$2^{70.8}$	$2^{108.4}$	-	$2^{164.9}$	-
2^{90}	128	16	$2^{120.1}$	$2^{109.1}$	-	-	$2^{108.6}$	$2^{70.8}$	$2^{108.5}$	-	$2^{168.0}$	-
2^{90}	128	20	2^{inf}	-	-	-	$2^{108.7}$	$2^{71.2}$	$2^{108.5}$	-	$2^{174.2}$	-
2^{90}	128	24	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{179.1}$	-
2^{90}	128	28	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{183.3}$	-
2^{90}	128	32	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{187.8}$	-
2^{90}	128	36	2^{inf}	-	-	-	$2^{109.0}$	$2^{71.2}$	$2^{108.6}$	-	$2^{191.3}$	-
2^{90}	256	10	$2^{91.2}$	$2^{76.2}$	$2^{312.5}$	$2^{30.9}$	$2^{169.8}$	$2^{112.2}$	$2^{204.1}$	-	-	-
2^{90}	256	12	$2^{126.2}$	$2^{126.2}$	-	-	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{90}	256	14	$2^{169.8}$	$2^{168.8}$	$2^{303.9}$	$2^{35.3}$	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{90}	256	16	$2^{213.0}$	$2^{206.0}$	-	-	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{90}	256	18	2^{inf}	-	$2^{265.7}$	$2^{60.0}$	$2^{169.9}$	$2^{111.8}$	$2^{204.1}$	-	-	-
2^{90}	256	20	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{90}	256	22	2^{inf}	-	$2^{255.7}$	$2^{64.0}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{90}	256	24	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{90}	256	26	2^{inf}	-	$2^{238.1}$	$2^{67.4}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{90}	256	28	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	-	-
2^{90}	256	30	2^{inf}	-	$2^{232.1}$	$2^{70.3}$	$2^{170.2}$	$2^{112.3}$	$2^{204.1}$	-	$2^{410.0}$	-
2^{90}	256	32	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{414.3}$	-
2^{90}	256	34	2^{inf}	-	$2^{167.5}$	$2^{87.5}$	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{417.6}$	-
2^{90}	256	36	2^{inf}	-	-	-	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{421.7}$	-
2^{90}	256	38	2^{inf}	-	$2^{158.5}$	$2^{90.2}$	$2^{170.2}$	$2^{112.3}$	$2^{204.2}$	-	$2^{425.1}$	-